Node.js — Day 13

HTTP, Your First Server, Requests, Responses & Basic Routing

Learning goal: Understand what an HTTP server is and build a simple server using Node.js's built-in **http** module.

> **Day 13 milestone:** You created and ran your first Node.js HTTP server and successfully saw its response at **localhost:3000**.

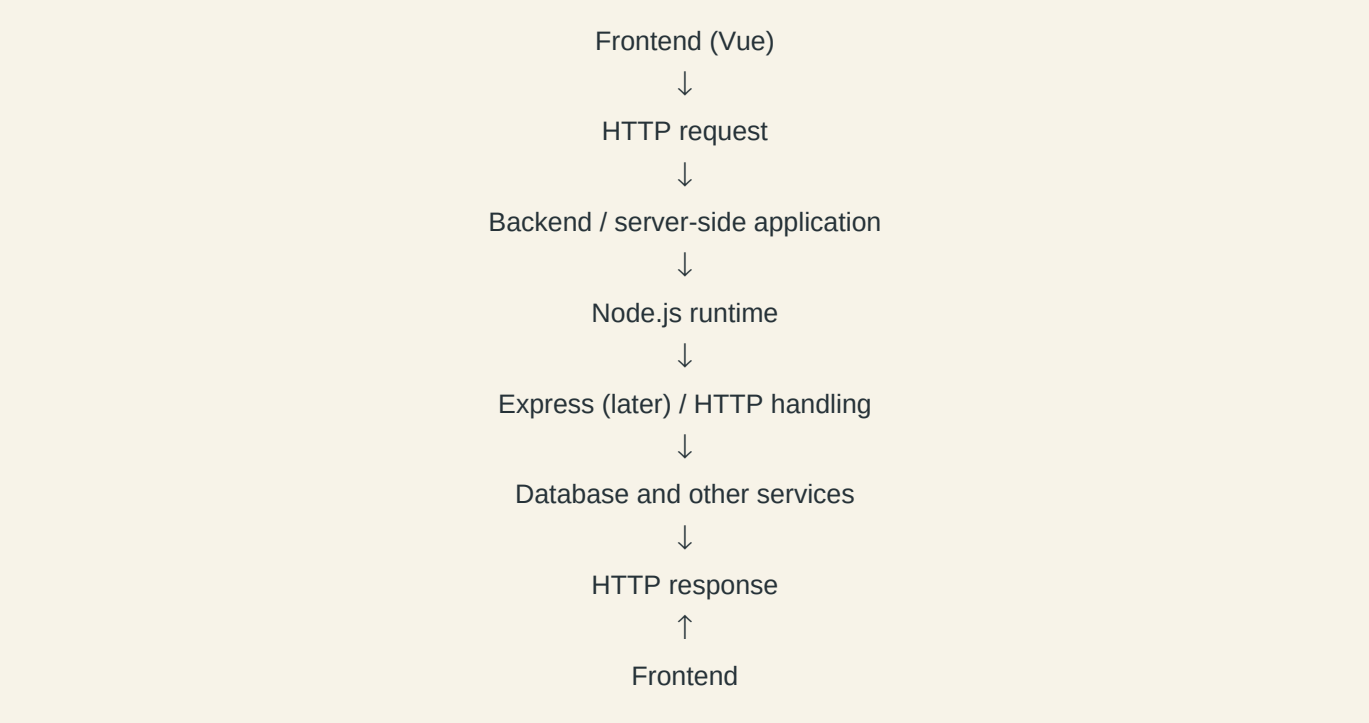
1. Backend vs Server vs Node.js vs Express

Backend is the server-side part of an application. It contains application logic, APIs, authentication, and database interaction.

Server is a program/computer that listens for requests and sends responses.

Node.js is a JavaScript runtime that lets JavaScript run outside the browser.

Express is a framework that runs on Node.js and makes building HTTP servers and APIs easier. We will learn it later.



2. The http Module

Node.js provides a built-in module called **http** that allows us to create an HTTP server.

```
const http = require("http");
```

It is built into Node.js, so it does not need to be installed with npm.

3. Your First HTTP Server

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.end("Hello from my server!");
});
```

```
server.listen(3000);
```

This creates a server and tells it to listen on port **3000**. When you open **http://localhost:3000**, the server sends the message back to the browser.

4. server.listen(3000)

```
server.listen(3000);
```

This tells Node.js to start listening for incoming requests on port 3000.

> **Important:** starting the server does not itself execute the request callback. The callback runs when an HTTP request arrives.

5. req and res

When a request arrives, Node.js gives your callback two objects:

```
(req, res) => {
  // req = request
  // res = response
}
```

req contains information about what the client/browser is requesting.

res is used to send the response back to the client.

6. Request → Response Cycle

Browser / Client
↓
HTTP Request
↓
Node.js HTTP Server
↓
(req, res) callback
↓
Server logic / routing
↓
res.end()
↓
HTTP Response
↓
Browser / Client

The callback runs **once for each incoming request**. It does not run once for req and once for res. If the browser makes two requests, the callback can run twice.

7. req.method

req.method tells you the HTTP method used by the client.

```
console.log(req.method);
```

When you enter a URL in a browser, the request is typically a **GET** request.

```
GET
```

Other methods such as POST become important when we learn REST APIs.

8. req.url

req.url tells you the requested path.

```
console.log(req.url);
```

For **http://localhost:3000/users**, the value is:

```
/users
```

Together, the method and URL describe the request:

```
GET /users
```

9. res.end()

res.end() sends/ends the response.

```
res.end("Hello!");
```

The text is sent back to the client, which can then display it.

10. Basic Routing with req.url

Routing means deciding what response to send based on the requested path.

```
const http = require("http");

const server = http.createServer((req, res) => {
  if (req.url === "/") {
    res.end("Home Page");
  }
  else if (req.url === "/about") {
    res.end("About Page");
  }
  else if (req.url === "/users") {
    res.end("Users Page");
  }
  else {
    res.end("Page Not Found");
  }
});

server.listen(3000);

/        → Home Page
/about   → About Page
/users   → Users Page
anything → Page Not Found
```

11. Practical Products Route

```
if (req.url === "/products") {
  res.end("Products available");
}
```

A request such as **GET /products** receives **Products available**.

12. Why This Matters for Express

We learn basic Node.js routing first so you understand what Express is simplifying.

```
Node.js:
if (req.url === "/users") {
  res.end("Users");
}

Express later:
app.get("/users", (req, res) => {
  res.send("Users");
});
```

The request/response idea stays the same; Express gives us cleaner tools for routing, middleware and APIs.

13. Important Rules

• **http** is a built-in Node.js module.

• **createServer()** creates an HTTP server.

• **listen(3000)** starts listening on port 3000.

• **req** represents the incoming request.

• **res** represents the outgoing response.

• **req.method** gives the HTTP method.

• **req.url** gives the requested path.

• **res.end()** sends/ends the response.

• The request callback runs once for each incoming request.

• Routing chooses a response based on the requested path.

14. Common Confusion

Node.js is not the same thing as a server. Node.js is the runtime. We can use Node.js's **http** module to create an HTTP server.

```
Node.js runtime
      ↓
http module
      ↓
HTTP server
      ↓
receives requests
      ↓
sends responses
```

15. Practice Completed

All Day 13 exercises were answered correctly. ■

Final score: 4/4 core questions correct, plus the practical routing exercises.

16. Quick Recap

http → built-in module for HTTP servers.

createServer() → creates the server.

listen() → starts listening for requests.

req → incoming request.

req.method → HTTP method.

req.url → requested path.

res → outgoing response.

res.end() → sends/ends the response.

Routing → different responses for different paths.

17. Key Takeaway

> **A client sends an HTTP request. A Node.js HTTP server receives it, uses req to understand it, runs server-side logic/routing, and uses res to send a response back. Express will later make this process much easier to organize.**